Rômulo Maurício Fonseca Júnior - 15553801
EP1 - Organização e Arquitetura de Computadores

As arquiteturas utilizadas neste trabalho serão:

ARM64 15.1.0

Nome completo: ARMv8-A AArch64
Tamanho da palavra: 64 bits
Denominação dos registradores:

- **Propósito geral:** `x0` a `x30` (64 bits)
- Versão de 32 bits dos mesmos registradores: `w0` a `w30`
-
- **Registradores especiais:**
  - `sp` = stack pointer (ponteiro de pilha)
  - `x29` = frame pointer (FP)
  - `x30` = link register (LR, endereço de retorno)

Parâmetros de função: passados nos registradores `x0`–`x7`
Retorno de função: registrador `x0` (inteiros) ou `d0` (ponto flutuante)
Referência oficial:
https://drive.google.com/file/d/19kkxKRHZRSwSY1n43ePaCfPsBg3LFfUB/view?usp=drive_link

RISC-V 32 bits

Nome completo: RISC-V RV32IM
(32 bits, com suporte a multiplicação/divisão)
Tamanho da palavra: 32 bits
Denominação dos registradores:

- **x0 a x31**, com apelidos de convenção ABI:
  - `x0` = zero (sempre 0)
  - `x1` = `ra` (return address)
  - `x2` = `sp` (stack pointer)
  - `x10` a `x17` = `a0` a `a7` (argumentos/funções)
  - `x5`–`x7` = `t0`–`t2` (temporários)
  - `x8` = `s0`/`fp` (frame pointer)

Parâmetros de função: `a0`–`a7`
Retorno de função: `a0`
Referência oficial:
https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.pdf

CÓDIGO P1.

Código C	Assembly ARM 64-bit (AArch64) GCC	Explicação (AArch64)	Assembly RISC-V 32-bit (RV32GC) GCC	Explicação (RV32
Setup da Função	stp x29, x30, [sp, -16]! mov x29, sp	Prólogo: 1. stp: Salva dois registradores na pilha (stack): x29 (Frame Pointer) e x30 (Link Register, que guarda o endereço de retorno). O ! indica que o ponteiro da pilha (sp) é atualizado. 2. mov: Define o novo Frame Pointer (x29) para o local atual da pilha.	addi sp, sp, -16 sw ra, 12(sp) sw s0, 8(sp) addi s0, sp, 16	Prólogo:1. addi: Aloca 16 bytes na pilha. 2. sw: Salva os registradores na pilha: ra (Return Address) e s0 (Frame Pointer). 3. addi: Define o novo Frame Pointer (s0) para o topo do quadro da pilha.
printf("Olá\n");	adrp x0, .LC0 add x0, x0, :lo12:.LC0	Carregando o Parâmetro:Para chamar printf, precisamos passar o endereço da string "Olá\n" como primeiro argumento. A convenção no ARM64 é usar o registrador x0. 1. adrp: Carrega a parte "alta" do endereço da string (.LC0) em x0. 2. add: Adiciona a parte "baixa" do endereço, completando o ponteiro para a string.	lui a0, %hi(.LC0) addi a0, a0, %lo(.LC0)	Carregando o Parâmetro:O primeiro argumento de uma função no RV32 é passado pelo registrador a0. 1. lui: Carrega a parte alta do endereço da string (.LC0) em a0. 2. addi: Adiciona a parte baixa para formar o endereço completo.
	bl printf	Chamada da Função:bl (Branch with Link) salta para a função printf e, ao mesmo	call printf	Chamada da Função:call é uma "pseudo-instrução" que geralmente se expande para

		tempo, salva o endereço da próxima instrução no registrador de link (x30) para saber para onde voltar.		auipc e jalr. Ela salta para a função printf e salva o endereço de retorno no registrador ra (Return Address).
} (Fim da Função)	nop ldp x29, x30, [sp], 16 ret	Epílogo:1. nop: Nenhuma operação. Muitas vezes inserida por alinhamento. 2. ldp: Restaura os registradores x29 e x30 com os valores salvos na pilha. 3. ret: Lê o endereço de retorno do registrador x30 e salta para ele.	li a0, 0 lw ra, 12(sp) lw s0, 8(sp) addi sp, sp, 16 ret	Epílogo:1. li: O compilador, por segurança, coloca 0 como valor de retorno em a0. 2. lw: Restaura os registradores ra e s0 da pilha. 3. addi: Libera o espaço alocado na pilha. 4. ret: É uma pseudo-instrução para jalr x0, 0(ra), que retorna ao chamador.
Dados Estáticos	.LC0: .string "Olá\n"	A string "Olá\n" é armazenada em uma seção de dados somente leitura do programa, identificada aqui pela etiqueta .LC0.	.LC0: .string "Ol\303\241\n"	A string é armazenada da mesma forma. Note que o caractere 'á' foi codificado em UTF-8 (C3 A1), o que é refletido no assembly como \303\241.

1. **Nomenclatura de Registradores:**
    - **AArch64:** Usa `x0`-`x30` para registradores de propósito geral. `x0` é o primeiro argumento e `x30` é o Link Register.
    - **RISC-V:** Usa `x0`-`x31`. Alguns têm nomes de ABI (Application Binary Interface) como `a0` (para `x10`, primeiro argumento) e `ra` (para `x1`, endereço de retorno).
2. **Carregamento de Endereços:**
    - **AArch64:** Usa `adrp` + `add` para carregar um endereço relativo ao PC (Program Counter), o que gera código que pode ser executado de qualquer lugar na memória (position-independent code).
    - **RISC-V:** Usa `lui` + `addi` para carregar um endereço absoluto de 32 bits em duas etapas.
3. **Instrução de Chamada:**

- **AArch64:** `bl` (Branch with Link) é a instrução nativa para chamadas de função.
- **RISC-V:** `call` é uma convenção (pseudo-instrução) que usa `jalr` (Jump and Link Register) para efetuar a chamada.

Ambas as arquiteturas seguem a filosofia **RISC**, onde as operações são mais simples e explícitas. Vemos isso na forma como ambas precisam de duas instruções para carregar um endereço de 32/64 bits em um registrador.

CÓDIGO P2

Código C	Assembly ARM 64-bit (AArch64)	Explicação (AArch64)	Assembly RISC-V 32-bit (RV32GC)	Explicação (RV32)

a=12;
b=11;
r=0;
	mov w0, 12
	str w0, [sp, 28]
	mov w0, 11
	str w0, [sp, 24]
	str wzr, [sp, 20]
		Inicialização:1. mov: Move os valores literais para um registrador temporário (w0). wzr é o registrador "zero".
		2. str: Armazena (store) os valores daquele registrador em locais na pilha ([sp, offset]).
			li a5, 12
			sw a5, 28(sp)
			li a5, 11
			sw a5, 24(sp)
			sw zero, 20(sp)
				Inicialização:1. li: Carrega (load immediate) os valores literais em um registrador (a5). zero é o registrador que sempre contém 0.
				2. sw: Armazena (store word) os valores na pilha.

if (b & 0x01)	ldr w0, [sp, 24] and w0, w0, 1 cbz w0, .L4	Teste do 1º Bit:1. ldr: Carrega o valor de b da pilha para w0. 2. and: Realiza a operação E bit a bit com 1. 3. cbz: Se o resultado for zero (w0 == 0), salta (branch) para a etiqueta .L4, pulando o bloco do if.	lw a5, 24(sp) andi a5, a5, 1 beqz a5, .L2	Teste do 1º Bit:1. lw: Carrega (load word) o valor de b da pilha para a5. 2. andi: Realiza a operação E bit a bit com o imediato 1. 3. beqz: Se o resultado for zero, salta (branch if equal to zero) para .L2.
r+=a;	ldr w1, [sp, 28] ldr w0, [sp, 20] add w0, w0, w1 str w0, [sp, 20]	Soma Condicional:1. ldr: Carrega a e r em registradores. 2. add: Soma r com a. 3. str: Salva o novo valor	lw a4, 28(sp) lw a5, 20(sp) addw a5, a5, a4 sw a5, 20(sp)	Soma Condicional:1. lw: Carrega a e r em registradores. 2. addw: Soma r com a. 3. sw: Salva o novo r na

		de r de volta na pilha.		pilha.
.L4: a=a<<1;	ldr w0, [sp, 28] lsl w0, w0, 1 str w0, [sp, 28]	Deslocamento à Esquerda:1. ldr: Carrega a da pilha. 2. lsl: Realiza o deslocamento lógico à esquerda (Logical Shift Left) por 1 bit. 3. str: Salva o novo valor de a.	.L2: lw a5, 28(sp) slli a5, a5, 1 sw a5, 28(sp)	Deslocamento à Esquerda:1. lw: Carrega a da pilha. 2. slli: Realiza o deslocamento lógico à esquerda (Shift Left Logical Immediate). 3. sw: Salva o novo valor de a.
(O padrão se repete para & 0x02, & 0x04, & 0x08)	...	...	...	...
printf("%d\n", r);	ldr w1, [sp, 20] adrp x0, .LC0 add x0, x0, :lo12:.LC0 bl printf	Chamada Final:1. ldr w1, [sp, 20]: Carrega o valor final de r da pilha para o registrador do 2º argumento (w1). 2. O resto carrega a string e chama printf.	lw a1, 20(sp) lui a0, %hi(.LC0) addi a0, a0, %lo(.LC0) call printf	Chamada Final:1. lw a1, 20(sp): Carrega o valor final de r da pilha para o registrador do 2º argumento (a1). 2. O resto carrega a string e chama printf.

Principais Diferenças

1. **Instruções de Acesso à Memória:** Ambas as arquiteturas usam o padrão "Load/Store", onde os dados precisam ser carregados da memória (pilha) para registradores antes de qualquer operação.
    - AArch64: `ldr` (Load Register) / `str` (Store Register).
    - RISC-V: `lw` (Load Word) / `sw` (Store Word).
2. **Instruções de Bits e Deslocamento:** As instruções são muito parecidas, refletindo a filosofia RISC de ambas.
    - **Bitwise AND:** `and` (AArch64) vs. `andi` (RV32).
    - **Shift Left:** `lsl` (AArch64) vs. `slli` (RV32).
3. **Salto Condicional:** O método para testar e saltar é ligeiramente diferente.
    - **AArch64:** Usa `cbz` (Compare and Branch on Zero), uma instrução que combina o teste e o salto em um único passo.
    - **RISC-V:** Usa `beqz` (Branch if Equal to Zero), que testa o valor de um registrador e salta se a condição for verdadeira.

CÓDIGO P3

Código C	Assembly ARM 64-bit (AArch64)	Explicação (AArch64)	Assembly RISC-V 32-bit (RV32GC)	Explicação (RV32)
Setup da Função e do Laçoint i=0;	stp x29, x30, [sp, -32]! mov x29, sp stp x19, x20, [sp, 16] mov w19, 0	Prólogo e Inicialização:1. O prólogo padrão da função salva os ponteiros na pilha (sp). 2. mov w19, 0: O contador i é inicializado com 0 diretamente no registrador w19. O compilador escolhe w19 por ser um registrador "callee-saved", ou seja, seu valor é preservado mesmo após a chamada de outras funções como printf.	addi sp, sp, -32 sw ra, 28(sp) sw s0, 24(sp) sw s1, 20(sp) li s1, 0	Prólogo e Inicialização:1. O prólogo salva os registradores necessários na pilha.2. li s1, 0: O contador i é inicializado com 0 no registrador s1. s1 também é um registrador "callee-saved", escolhido pelo mesmo motivo que w19 no ARM.
Início do Laçodo {	.L3:	Etiqueta do Laço:Esta é a etiqueta (label) que marca o início do corpo do laço. A instrução de salto no final do laço voltará para este ponto.	.L3:	Etiqueta do Laço:Similarmente, esta etiqueta marca o ponto de entrada do laço para o qual a instrução de desvio condicional retornará.
printf("%d", i);	adrp x0, .LC0 add x0, x0, :lo12:.LC0 mov w1, w19 bl printf	Chamada da Função:1. adrp/add: Carregam o endereço da string de formato "%d" no registrador do 1º argumento (x0). 2. mov w1, w19: Move o valor de i (que está em w19) para w1, o registrador do 2º argumento. 3. bl printf: Chama a função printf.	lui a0, %hi(.LC0) addi a0, a0, %lo(.LC0) mv a1, s1 call printf	Chamada da Função:1. lui/addi: Carregam o endereço da string "%d" no 1º argumento (a0). 2. mv a1, s1: Move (move) o valor de i (de s1) para o 2º argumento (a1). mv é uma pseudo-instrução para addi a1, s1, 0. 3. call printf: Chama a função.
} while (++i<10);	add w19, w19, 1 cmp w19, 10 b.lt .L3	Incremento, Comparação e Salto:1. add w19,	addi s1, s1, 1 li a5, 10 blt s1, a5, .L3	Incremento, Comparação e Salto:1. addi s1, s1,

		w19, 1: Incrementa i (++i). 2. cmp w19, 10: Compara o novo valor de i com 10. 3. b.lt .L3: Se i for menor que (Less Than) 10, salta de volta para a etiqueta .L3, executando o laço novamente.		1: Incrementa i. 2. li a5, 10: Carrega o valor 10 em um registrador temporário (a5) para a comparação. 3. blt s1, a5, .L3: Se i (em s1) for menor que (Branch if Less Than) o valor em a5, salta de volta para .L3.
Fim da Função	(Epílogo)ldp x19, x20, [sp, 16] ldp x29, x30, [sp], 32 ret	Epílogo:Restaura os registradores salvos da pilha e retorna da função main.	(Epílogo)lw ra, 28(sp) lw s0, 24(sp) lw s1, 20(sp) addi sp, sp, 32 ret	Epílogo:Restaura os registradores da pilha, libera o espaço alocado e retorna.

Principais Diferenças e Similaridades

1. **Estrutura do Laço Eficiente:** Ambas as arquiteturas traduzem o `do-while` de forma muito natural e eficiente. A estrutura é basicamente: "faça o trabalho, depois incremente, compare e, se a condição for válida, salte para o início". Isso evita o salto incondicional que às vezes é visto no final de um laço `for` ou `while`.
2. **Uso de Registradores "Callee-Saved":** Como o valor de `i` precisa sobreviver à chamada da função `printf` (que poderia usar qualquer registrador "caller-saved" para seus próprios cálculos), o compilador alocou `i` em um registrador que `printf` tem a obrigação de preservar.
    - AArch64: `w19`
    - RISC-V: `s1`
3. **Instruções de Comparação e Salto:** A principal diferença na lógica do laço está aqui.
    - **AArch64:** Realiza a comparação e o salto em duas instruções separadas: `cmp` (que define flags de status) e `b.lt` (que lê essas flags). A instrução `cmp` pode usar um valor imediato (o número `10`), tornando o código mais compacto.
    - **RISC-V:** Precisa primeiro carregar o valor `10` em um registrador (`li a5, 10`) antes de poder comparar e saltar com uma única instrução (`blt`). A instrução `blt` compara os dois registradores e salta, tudo em um passo.

CÓDIGO P4

Código C	Assembly ARM 64-bit (AArch64)	Explicação (AArch64)	Assembly RISC-V 32-bit (RV32GC)	Explicação (RV32GC)
double f(double x, double y, double z)	(entrada da função)	Passagem de Parâmetros: Os três argumentos double (x, y, z) são passados através dos primeiros três registradores de ponto flutuante: d0, d1, e d2.	(entrada da função)	Passagem de Parâmetros: Os argumentos double são passados nos registradores de ponto flutuante: fa0 (x), fa1 (y), e fa2 (z). Estes são registradores de 64 bits fornecidos pela extensão 'D'.
return x*x	fmul d3, d0, d0	x*x: A instrução fmul (Floating-Point Multiply) multiplica o valor no registrador d0 por ele mesmo e armazena o resultado em um registrador de destino, d3.	fmul.d fa5, fa0, fa0	x*x: A instrução fmul.d (Floating-Point Multiply Double) faz a multiplicação de fa0 com fa0 e guarda o resultado em fa5.
+y*y	fmul d4, d1, d1\nfadd d3, d3, d4	+y*y: 1. fmul: Calcula y*y (de d1) e armazena em d4. 2. fadd: Soma (Floating-Point Add) o resultado anterior (d3, que continha x*x) com o novo (d4), atualizando d3.	fmul.d fa6, fa1, fa1\nfadd.d fa5, fa5, fa6	+y*y: 1. fmul.d: Calcula y*y (de fa1) e armazena em fa6. 2. fadd.d: Soma (Floating-Point Add Double) o acumulado em fa5 com fa6.
+z*z;	fmul d1, d2, d2\nfadd d0, d3, d1	+z*z: 1. fmul: Calcula z*z (de d2). 2. fadd: Soma o resultado acumulado (d3) com o último quadrado. O resultado final é colocado em d0.	fmul.d fa7, fa2, fa2\nfadd.d fa0, fa5, fa7	+z*z: 1. fmul.d: Calcula z*z (de fa2). 2. fadd.d: Soma o acumulado (fa5) com o último quadrado, colocando o resultado final em fa0.
}	ret	Retorno: O valor de retorno de uma função double é, por convenção, esperado no registrador d0. A instrução ret então	ret	Retorno: Por convenção, o valor de retorno double é colocado em fa0. A instrução ret retorna ao chamador.

		retorna ao chamador.		

Ambas as arquiteturas possuem um conjunto de **registradores e instruções específicas para ponto flutuante**. Isso é crucial para a performance, pois cálculos com `double` são complexos e seriam muito lentos se emulados com instruções de inteiros ("soft-float").

- **AArch64:** Registradores `d0`-`d31` e instruções como `fmul`, `fadd`.
- **RISC-V (com extensão 'G' ou 'D'):** Registradores `fa0`-`fa7` (argumentos/retorno) e instruções como `fmul.d`, `fadd.d`.

Convenção de Chamada (ABI): A forma de passar e retornar argumentos é muito similar. Os primeiros argumentos de ponto flutuante vão em registradores de ponto flutuante (`d0`-`d7` no ARM, `fa0`-`fa7` no RISC-V), e o valor de retorno também usa o primeiro desses registradores (`d0`/`fa0`).
Filosofia RISC: A abordagem é a mesma: carregar os dados em registradores e usar instruções simples e rápidas (`fmul`, `fadd`) para realizar o trabalho passo a passo.

CÓDIGO P5

Código C	Assembly ARM 64-bit (AArch64)	Explicação (AArch64)	Assembly RISC-V 32-bit (RV32GC)	Explicação (RV32GC)
int a[]={...};	.section .data\na:\n.word 3, 5, 7, 11, 13	Definição dos Dados: O array a é alocado na seção de memória de dados (.data) do programa, já com seus valores iniciais.	.section .data\na:\n.word 3, 5, 7, 11, 13	Definição dos Dados: A abordagem é idêntica. O array é colocado na seção .data com seus valores.
Setup do Laço, int i=0;	mov w19, 0\nadrp x20, a\nadd x20, x20, :lo12:a	Inicialização: 1. mov w19, 0: O contador i é zerado no registrador w19. 2. adrp/add: O endereço base do array a é carregado no registrador x20.	li s1, 0\nlui s2, %hi(a)\naddi s2, s2, %lo(a)	Inicialização: 1. li s1, 0: O contador i é zerado no registrador s1. 2. lui/addi: O endereço base de a é carregado no registrador s2.
Início do Laço, do {	.L3:	Etiqueta do Laço: Marca o início do corpo do laço.	.L5:	Etiqueta do Laço: Marca o ponto de entrada do laço.
printf("%d, %d", i, a[i]);	mov w1, w19\nlsl x9, x19, 2\nldr w2,	Cálculo de Endereço e Carga:	mv a1, s1\nslli a5, s1, 2\nadd a5, s2,	Cálculo de Endereço e Carga:

	[x20, x9]	1. mov w1, w19: O 2º argumento de printf (i) é colocado em w1. 2. lsl x9, x19, 2: Calcula o deslocamento. Multiplica i por 4. 3. ldr w2, [x20, x9]: Carrega o valor do endereço base + offset para w2.	a5\nlw a2, 0(a5)	1. mv a1, s1: O argumento i é colocado em a1. 2. slli a5, s1, 2: Calcula o deslocamento. 3. add a5, s2, a5: Soma a base com o deslocamento. 4. lw a2, 0(a5): Carrega o valor daquele endereço para a2.
(chamada printf)	adrp x0, .LC0\n...\nbl printf	Chamada da Função: Carrega a string de formato e chama printf. Os argumentos já estão nos registradores corretos (x0, w1, w2).	lui a0, %hi(.LC0)\n...\ncall printf	Chamada da Função: Carrega a string e chama printf. Os argumentos estão em a0, a1, a2.
} while (++i<5);	add w19, w19, 1\ncmp w19, 5\nb.ne .L3	Condição do Laço: 1. add: Incrementa i. 2. cmp: Compara i com 5. 3. b.ne: Salta de volta para .L3 se i for diferente de 5.	addi s1, s1, 1\nli a5, 5\nblt s1, a5, .L5	Condição do Laço: 1. addi: Incrementa i. 2. li: Carrega 5 em um registrador. 3. blt: Salta de volta para .L5 se i for menor que 5.

- A **Arch64** possui **modos de endereçamento mais complexos**. A instrução `ldr w2, [x20, x9]` consegue usar o valor de dois registradores (base + offset) para encontrar o endereço de memória em um único passo. Isso é um resquício de designs CISC que foi mantido por sua eficiência.
- **RISC-V** segue a filosofia RISC de forma mais pura. O cálculo é feito em passos explícitos e separados: primeiro o deslocamento (`slli`), depois a soma do endereço base com o deslocamento (`add`), e finalmente o carregamento do dado daquele endereço final (`lw`).

Ambos chegam ao mesmo resultado, mas o AArch64 consegue ser um pouco mais denso e expressivo para operações de acesso à memória indexada.

CÓDIGO P6

Seção	Código C	Assembly ARM 64-bit (AArch64)	Explicação (AArch64)	Assembly RISC-V 32-bit (RV32GC)	Explicação (RV32GC)

Função main	int a[]={...};	sub sp, sp, #32\nmov w9, 3\nstr w9, [sp, 12]\n...	Alocação e Inicialização: 1. sub: Aloca 32 bytes na pilha. 2. mov/str: Os valores do array são armazenados um a um na pilha.	addi sp, sp, -32\nli a5, 3\nsw a5, 12(sp)\n...	Alocação e Inicialização: 1. addi: Aloca espaço na pilha. 2. li/sw: Carrega cada valor do array e o armazena na pilha.
Função main	p(n,a);	mov w0, 5\nadd x1, sp, 12\nbl p	Preparando Parâmetros e Chamando: 1. mov w0, 5: O 1º argumento (n) é colocado em w0. 2. add x1, sp, 12: O 2º argumento (a) é um ponteiro para o array na pilha, colocado em x1. 3. bl p: Chama a função p().	li a0, 5\naddi a1, sp, 12\ncall p	Preparando Parâmetros e Chamando: 1. li a0, 5: O 1º argumento (n) é colocado em a0. 2. addi a1, sp, 12: O ponteiro a (endereço na pilha) é calculado e colocado em a1. 3. call p: Chama a função p().
Função p	void p (int n, int *a)	mov w20, w0\nmov x19, x1	Recebendo Parâmetros: A função assume que n está em w0 e *a está em x1. Ela os move para registradores callee-saved (w20, x19) para uso seguro.	mv s0, a0\nmv s1, a1	Recebendo Parâmetros: Os parâmetros n (de a0) e *a (de a1) são movidos para os registradores callee-saved s0 e s1.
Função p	int i=0;\ndo { ... }	mov w21, 0\n.L8:	Início do Laço: O contador i é inicializado em w21. A etiqueta .L8 marca o início do laço.	li s2, 0\n.L4:	Início do Laço: O contador i é zerado em s2. A etiqueta .L4 marca o início.
Função p	a[i]	lsl x9, x21, 2\nldr w2, [x19, x9]	Acesso ao Ponteiro: A lógica é a mesma do exercício anterior, mas agora a base do cálculo é o	slli a5, s2, 2\nadd a5, s1, a5\nlw a2, 0(a5)	Acesso ao Ponteiro: O cálculo usa o ponteiro 'a' do registrador s1 como base para encontrar o elemento a[i].

			ponteiro 'a' recebido no registrador x19.		
Função p	} while (++i<n);	add w21, w21, 1\ncmp w21, w20\nb.lt .L8	Condição do Laço: O contador i (w21) é incrementado e comparado com n (w20). O salto (b.lt) volta se a condição for verdadeira.	addi s2, s2, 1\nblt s2, s0, .L4	Condição do Laço: O contador i (s2) é incrementado e comparado com n (s0). O salto (blt) executa a mesma lógica.

Este último exemplo amarra tudo:

1. **Arrays Locais Vivem na Pilha:** Diferente do array global do exercício anterior (que ficava na seção `.data`), um array local como `a` é criado na pilha (`stack`) quando a função `main()` é chamada.
2. **Passagem de Ponteiros:** Quando se passa um array para uma função, não se copia o array inteiro. Apenas o **endereço de memória** do primeiro elemento é passado. Vimos isso claramente nas instruções `add x1, sp, 12` (ARM) e `addi a1, sp, 12` (RISC-V).
3. **Inlining é uma Otimização Crucial:** Compiladores modernos evitam o custo de uma chamada de função sempre que possível, tornando o código final mais rápido.